



Computer Vision and Image Processing

Lecture 13

Features

- **Feature point / keypoint / interest point:** A specific location (x,y) in the image where something "interesting" happens.
- **Feature descriptor:** A numerical vector that encodes what the neighborhood around that point looks like.

When someone says "*We detected 500 features in this image,*" they usually mean **keypoints**. When they say "*We matched features between two images,*" they mean **descriptors**.

Features

In computer vision, when we look at two images of the same scene (e.g., taken from different angles or lighting conditions), how do we know they match?

- **Global features** (like color histograms) look at the whole image and fail if there's occlusion or a viewpoint change.
- **Local features** focus on distinct, invariant patches (like corners, blobs, or distinctive textures) that can be reliably found and tracked across different images.

Features

A good feature should ideally be:

- **Distinctive** – easy to distinguish from other features.
- **Repeatable** – detected again if the same object appears in another image.
- **Robust** – should remain useful under:
 - Rotation
 - Scale changes
 - Illumination changes
 - Noise
 - Small viewpoint changes

Features

The Pipeline

- **Detection:** Find interesting points (keypoints/corners).
- **Description:** Create a mathematical fingerprint (descriptor) for the local neighborhood around each keypoint.
- **Matching:** Compare descriptors between Image A and Image B to find correspondences.
- **Verification:** Use geometric constraints (like RANSAC) to throw out false matches. *(Focusing here on steps 1–3).*

Feature Detection Algorithms

The detector identifies the location of interesting points

1. Harris Corner Detector
2. Shi-Tomasi
3. FAST (Features from Accelerated Segment Test)
4. SIFT Detector (Scale-Invariant Feature Transform)
5. SURF Detector (Speeded-Up Robust Features)

Feature Descriptors Algorithms

For each keypoint, we generate a numerical representation called a **feature descriptor**. The descriptor is essentially the **signature** of that local feature. A **feature descriptor** is a numerical representation of the visual characteristics of a local image region.

1. SIFT (Scale-Invariant Feature Transform)
2. SURF (Speeded-Up Robust Features)
3. ORB (Oriented FAST and Rotated BRIEF)
4. HOG (Histogram of Oriented Gradients)

Feature Matching

Feature matching is the process of establishing correspondences between the local features (keypoints and their descriptors) of two or more different images of the same scene or object.

Once you have detected keypoints and generated their mathematical "fingerprints" (descriptors, like SIFT or ORB) in **Image A** and **Image B**, feature matching is how your algorithm figures out which descriptor in Image A belongs to the exact same physical point in Image B.

Feature Matching

Distance Metrics

- **Euclidean Distance (L_2):** Standard for floating-point descriptors like SIFT.
- **Hamming Distance:** Used for binary descriptors (like BRIEF, ORB) where features are strings of bits (counted via XOR + bit-counting).
-

Feature Matching

Matching Strategies

1. Brute-Force Matching

- Compare every descriptor in Image A with every descriptor in Image B. Find the closest one based on distance.
- *Drawback:* Prone to false positives because ambiguous features might accidentally find a "closest" neighbor that isn't actually a true match.

Feature Matching

2. Lowe's Ratio Test (The Gold Standard)

David Lowe introduced a brilliant heuristic in his SIFT paper to filter out bad matches:

- For a descriptor in Image A, find the **1st nearest neighbor** (NN_1) and the **2nd nearest neighbor** (NN_2) in Image B.

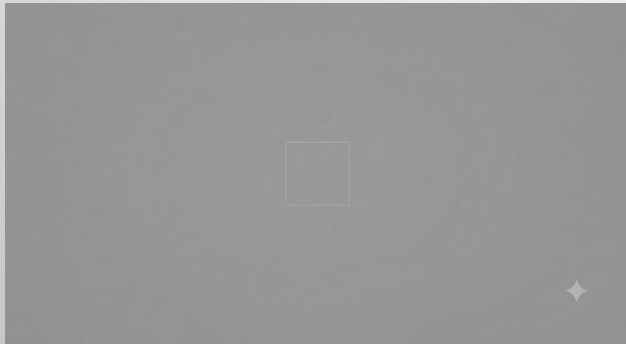
Compute the ratio:

- $$\text{Ratio} = \frac{\text{Distance}(NN_1)}{\text{Distance}(NN_2)}$$

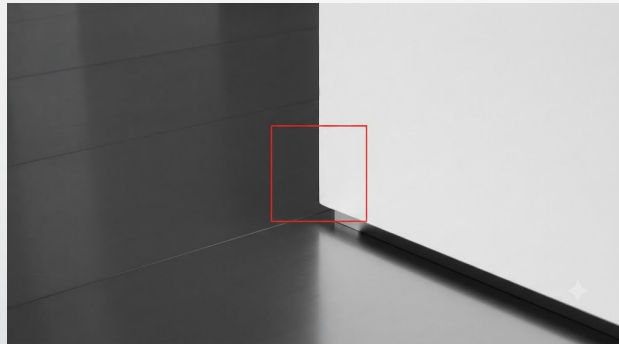
- **Intuition:** If NN_1 is truly distinct, its distance will be much smaller than NN_2 (ratio $\ll 1$). If the point is ambiguous, NN_1 and NN_2 will have very similar distances (ratio ≈ 1).
- **Threshold:** Typically, discard matches where the ratio is greater than 0.7 or 0.8.

Harris Corner Detection

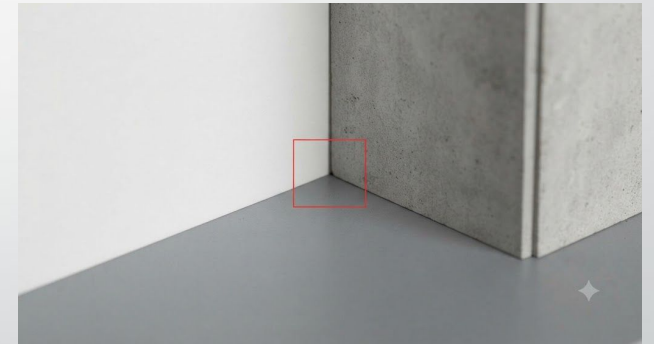
Developed by **Chris Harris and Mike Stephens** in **1988**, this detector relies on the idea of shifting a small window over the image and measuring the intensity change.



Flat

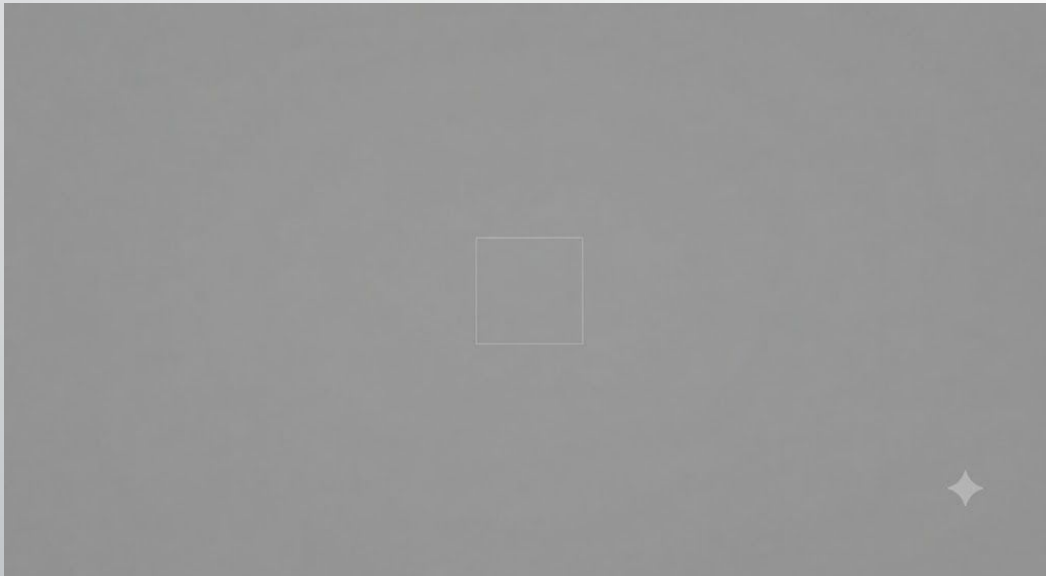


Edge



Corner

Harris Corner Detection

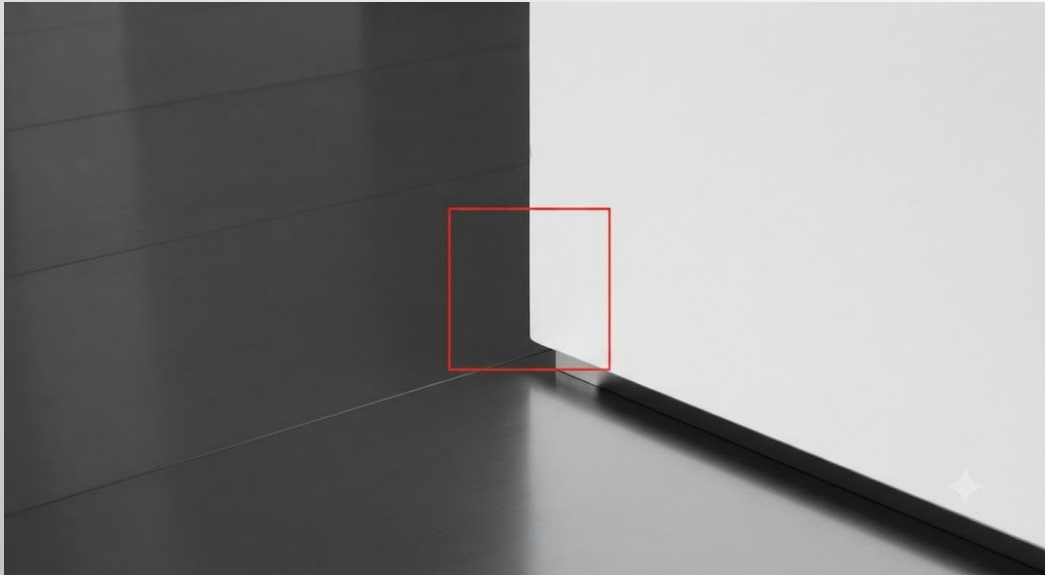


If we move a small window in any direction, the appearance does not change much.

Therefore:

- No significant intensity change
- This is not a useful feature.

Harris Corner Detection

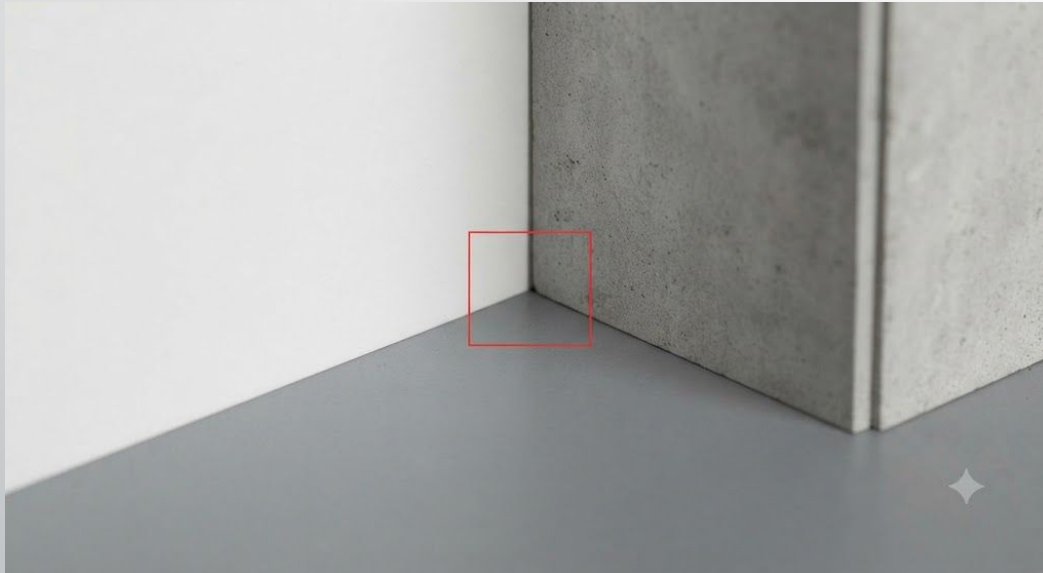


If we move the window:

- Along the edge → little change
- Across the edge → significant change

Therefore, an edge has strong intensity variation in **one direction**.

Harris Corner Detection



If we move the window:

- Horizontally → significant change
- Vertically → significant change

Therefore, a corner has strong intensity variation in **two directions**.

This is the key idea behind Harris Corner Detection.

Harris Corner Detection

- Suppose we take a small window around a pixel: $W(x, y)$
- Now we shift the window by: (u, v)
- We compare the original window with the shifted window.
- The change can be represented using the **sum of squared differences**:

Harris Corner Detection

- $$E(u, v) = \sum_{x, y} W(x, y) [I(x + u, y + v) - I(x, y)]^2$$

Where:

- $I(x, y)$ = image intensity
- $W(x, y)$ = weighting function
- (u, v) = displacement

The interpretation is:

- If $E(u, v)$ is small for every direction $\rightarrow$ flat region
- If $E(u, v)$ is large in one direction $\rightarrow$ edge
- If $E(u, v)$ is large in all directions $\rightarrow$ corner

Harris Corner Detection

Using a first-order Taylor approximation:

$$I(x + u, y + v) \approx I(x, y) + I_x u + I_y v$$

Where:

•

and

$$I_x = \frac{\partial I}{\partial x}$$

$$I_y = \frac{\partial I}{\partial y}$$

Harris Corner Detection

- Substituting into the error function: $E(u, v) \approx \sum W(x, y)(I_x u + I_y v)^2$
- This can be written in matrix form:

$$E(u, v) = [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

Where M is the **second-moment matrix**, also called the **structure tensor**:

$$M = \sum W(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- Usually, the matrix is written as:

$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

Harris Corner Detection

- The matrix M has two eigenvalues: λ_1, λ_2
- These eigenvalues describe the amount of intensity variation in two principal directions.
- **Flat Region:** $\lambda_1 \approx 0, \lambda_2 \approx 0$ (Very little change in any direction)
- **Edge:** $\lambda_1 \gg 0, \lambda_2 \approx 0$ (Strong variation in one direction)
- **Corner:** $\lambda_1 \gg 0, \lambda_2 \gg 0$ (Strong variation in both directions)

Harris Corner Detection

- Instead of explicitly calculating the eigenvalues, Harris proposed the response function:

$$R = \det(M) - k(\text{trace}(M))^2$$

Where:

$$\det(M) = \lambda_1 \lambda_2$$

and:

$$\text{trace}(M) = \lambda_1 + \lambda_2$$

- The parameter k is typically:

$$k \approx 0.04 \text{ to } 0.06$$

Harris Corner Detection

Instead of explicitly calculating the eigenvalues,
Harris proposed the response function:

$$R = \det(M) - k(\text{trace}(M))^2$$

Where:

$$\det(M) = \lambda_1 \lambda_2$$

and:

$$\text{trace}(M) = \lambda_1 + \lambda_2$$

The parameter k is typically:

$$k \approx 0.04 \text{ to } 0.06$$

$R > 0 \rightarrow \text{Corner}$

$R < 0 \rightarrow \text{Edge}$

$R \approx 0 \rightarrow \text{Flat region}$

Harris Corner Detection Algorithm

- Step 1: Convert Image to Grayscale
- Step 2: Compute Image Gradients: I_x and I_y by Usually using Sobel operators:
 - $I_x = I * S_x$
 - $I_y = I * S_y$
- Step 3: Compute Products: I_x^2 , I_y^2 , $I_x I_y$
- Step 4: Apply Gaussian Smoothing: The gradient products are averaged over a neighborhood: $\sum I_x^2 \sum I_y^2 \sum I_x I_y$ (This produces the matrix M)

Harris Corner Detection Algorithm

- Step 5: Calculate Harris Response: $R = \det(M) - k(\text{trace}(M))^2$
- Step 6: Thresholding: Only points satisfying: $R > T$ are considered corners.
- Step 7: Non-Maximum Suppression: If several nearby pixels are detected as corners, retain only the strongest one. This prevents a single physical corner from producing many redundant keypoints.

Limitation of Harris Corner Detection Algorithm

Harris corner detection is:

- **Translation invariant:** If the image moves, the corner can still be detected.
- **Rotation invariant:** The corner response is generally stable under rotation.
- However, standard Harris detection is **not inherently scale invariant:** The same corner may appear at different scales, making direct comparison difficult.
- This motivated the development of more advanced feature detectors and descriptors such as:
 - SIFT
 - SURF



The End